

Cheatsheet — Neovim & tmux · config de accel

NEOVIM

Leader = `Espacio`. La mayoría de comandos empiezan con Espacio + una letra que recuerda la acción. **Tip:** presiona `Espacio` y espera un momento → **which-key** te muestra un menú con todas las opciones disponibles.

<code>e</code> · Explorer (árbol de archivos)	<code>f</code> · Find (búsqueda difusa)	<code>s</code> · Split (ventanas)
<code>t</code> · Tab / Terminal	<code>h</code> · Hunk (cambios de git)	<code>x</code> · Trouble (errores)
<code>m</code> · forMat · <code>l</code> · Lint	<code>w</code> · Workspace (sesión)	<code>g</code> · Goto · <code>r/c</code> · rename/code

General & Movimiento	
<code>kj</code> / <code>jk</code>	Salir a modo normal estando en insert Más rápido que estirar la mano al Esc
<code>u</code> <code>nh</code>	Quitar el resaltado de la última búsqueda nh = No Highlight. Tras buscar con / queda todo pintado; esto lo limpia
<code>gb</code>	Volver atrás en el historial de saltos Regresa al lugar donde estabas antes de un salto (gd, búsqueda, etc.)
<code>u</code> <code>+</code> / <code>u</code> <code>-</code>	Incrementar / decrementar el número bajo el cursor

Portapapeles		solo lo que copias con y va al portapapeles del sistema
<code>y</code> <code>yy</code>	Copiar (yank) → portapapeles de macOS Lo pegas con Cmd+V en cualquier otra app	
<code>p</code>	Pegar el portapapeles Sirve para pegar tanto tus yanks como lo que copiaste fuera con Cmd+C	
<code>d</code> <code>c</code> <code>x</code>	Borrar / cambiar SIN tocar el portapapeles Van al "black hole": borras cosas sin perder lo que tenías copiado	
<code>u</code> <code>d</code>	Cortar Sí enviando al portapapeles ⚠ En archivos con LSP esta tecla muestra el diagnóstico de la línea (gana el LSP)	
<code>p</code> (visual)	Pegar encima de una selección sin perder lo copiado	

Ventanas / Splits		s = Split
<code>u</code> <code>sv</code>	Split vertical (dos paneles lado a lado)	
<code>u</code> <code>sh</code>	Split horizontal (uno arriba, otro abajo)	
<code>u</code> <code>se</code>	Igualar el tamaño de los splits	
<code>u</code> <code>sx</code>	Cerrar el split actual	
<code>u</code> <code>sm</code>	Maximizar / restaurar el split (foco temporal)	
<code>u</code> <code>sr</code>	Modo resize incremental: luego tap <code>h/l</code> (ancho) o <code>j/k</code> (alto), repetible; otra tecla sale Ideal para agrandar nvim-tree: <code>_sr</code> y tap <code>lll</code>	
<code>^h</code> <code>^j</code> <code>^k</code> <code>^l</code>	Moverte entre splits (y entre panes de tmux) Mismas teclas funcionan en vim y en tmux gracias a vim-tmux-navigator	

Pestañas & Terminal		t = Tab
 to	Abrir pestaña nueva (to = tab open)	
 tx	Cerrar pestaña	
 tn /  tp	Siguiente / anterior pestaña	
 tf	Abrir el buffer actual en una pestaña nueva	
 	Alternar entre el buffer actual y el anterior La barra de arriba (bufferline) muestra un tab por archivo abierto	
 bm	Buffers sin guardar (:ls +) ·  :ls = todos El + en la lista marca los modificados. Salta con :b N	
terminal	Para una terminal usa un pane de tmux (Ctrl+j al de abajo), no una terminal interna de nvim	

Explorador de archivos (nvim-tree)		e = Explorer
 ee	Abrir / cerrar el árbol de archivos	
 ef	Ubicar el archivo actual dentro del árbol	
 ea	Enfocar el árbol (saltar el cursor ahí)	
 ec /  er	Colapsar todo · Refrescar	
 ed	Duplicar el archivo seleccionado	

Dentro del árbol		teclas cuando el cursor está en el explorador
	Crear archivo o carpeta Termina el nombre en / para crear carpeta (ej. components/)	
 	Renombrar · Borrar (va a la Papelera)	
  	Cortar · Copiar · Pegar archivos	
   	Copiar al portapapeles: y = nombre (index.js) · Y = ruta relativa (src/index.js) · gy = ruta absoluta (/Users/.../src/index.js) · ge = nombre sin extensión (index)	
   	Abrir · en split vertical · en split horizontal · en pestaña nueva	
  	Previsualizar sin salir del árbol · ir a la carpeta padre · subir un nivel de raíz	
   	Ver ocultos (dotfiles) · ver git-ignored · Expandir todo · Colapsar todo	
	Ayuda: lista TODOS los atajos del árbol	

Búsqueda (Telescope)

f = Find

 ff	Buscar archivos por nombre en el proyecto Escribe parte del nombre; es difuso (no necesita ser exacto)
 fr	Archivos recientes (los que abriste últimamente)
 fb	Buffers abiertos (lista difusa de lo que tienes abierto)
 fs	Buscar texto dentro de todos los archivos (live grep) Busca una palabra/frase en TODO el proyecto en tiempo real
 fc	Buscar la palabra bajo el cursor en el proyecto
 fa	Búsqueda amplificada solo en los archivos abiertos
 ^j ^k 	Dentro del buscador: bajar / subir (navegar) · abrir el resaltado ^j/^k navegan (NO uses Tab para eso)
Tab  ^q	Marcar varios (multi-selección, sale un +) →  los manda al quickfix Para refactors: marca los usos y recórrelos con <code>__xq / :cnext</code>

LSP: ¿cuál uso? (definición vs declaración vs referencias...)

Definición (gd): te lleva a donde algo está **escrito/implementado de verdad**. Ej: sobre `miFuncion()` → salta al cuerpo de esa función. **Es el que más usarás.**

Declaración (gD): dónde se **declara** (se anuncia que existe) sin necesariamente implementarlo. En JS/TS suele coincidir con la definición; se nota más en C/C++ (te lleva al `.h`).

Referencias (gr): **todos los lugares donde se usa** ese símbolo en el proyecto. Ideal para "¿qué se rompe si cambio esto?".

Implementaciones (gi): para **interfaces / clases abstractas**: dónde se implementan concretamente. Ej: una interfaz `Repository` → sus clases que la cumplen.

Definición de tipo (gt): te lleva a la **definición del tipo** de una variable. Ej: sobre `const u: User` → salta a `interface User`.

Hover (K): muestra **documentación y tipo** en un popup, sin moverte del lugar.

Servidor TS: tsgo (TypeScript 7 nativo en Go, preview — más rápido, soporta monorepos). Para volver al clásico `ts_ls`: `use_tsgo = false` en `lspconfig.lua`. Node (`fs`, `path`...) autocompleta si hay `@types/node` en la carpeta padre.

LSP: navegación por el código

g = Goto

gd	Ir a la definición (dónde se implementa)
gD	Ir a la declaración
gr	Ver referencias (todos los usos) — dispara directo Se quitaron los defaults <code>grr/gmr/gra/gri/grt</code> de nvim 0.11 que hacían esperar el menú
gi	Ver implementaciones (de una interfaz)
gt	Ir a la definición del tipo
K	Documentación / tipo en popup (hover, con borde)
^s	Signature help: los parámetros de la función Útil al escribir la llamada; funciona en normal e insert

LSP: acciones y diagnósticos

c=code · r=rename · d=diagnostic

 ca	Code actions: arreglos que ofrece el editor Auto-importar, corregir error, refactor rápido, etc.
 rn	Renombrar símbolo en todo el proyecto (smart rename) Cambia la variable/función en todos sus usos a la vez
 rs	Reiniciar el LSP (si se cuelga o deja de sugerir)
 d	Ver el error/warning de la línea (popup)
 D	Lista de todos los diagnósticos del archivo
 [d] / [l]d	Saltar al diagnóstico anterior / siguiente
 ih	Inlay hints: mostrar/ocultar tipos inline Anotaciones grises de tipos y nombres de parámetro (i=inlay h=hints). Vienen encendidos por defecto

Git — cambios por bloque (gitsigns)

h = Hunk

<i>¿hunk?</i>	Un hunk es un bloque de líneas consecutivas modificadas respecto al último commit.
]h /  [h	Saltar al hunk siguiente / anterior
 hs /  hr	Stage / Reset del hunk stage = marcarlo para el commit · reset = descartar el cambio
 hS /  hR	Stage / Reset del archivo entero
 hp	Previsualizar el hunk (qué cambió)
 hu	Deshacer el último stage
 hb /  hB	Blame de la línea (quién la cambió) · toggle blame permanente
 hd	Diff del archivo vs. git
 lg	Abrir LazyGit: interfaz completa de git (commit, push, ramas...)

Trouble — panel de errores

x

 xx	Abrir/cerrar el panel Trouble
 xd /  xw	Diagnósticos del documento / de todo el workspace
 xq /  xl	Lista quickfix / loclist en formato bonito
 xt	TODOs del proyecto (TODO, FIXME, HACK...)

Formato & Lint

m = forMat · l = Lint

 mp	Formatear el archivo o la selección Usa prettier (JS/TS/JSON/YAML/MD) o stylua (Lua)
 l	Lint ahora (eslint_d) — buscar problemas de estilo/errores
<i>automático</i>	Se formatea solo al guardar y se hace lint al guardar / salir de insert

Sesiones de trabajo (auto-session)		w = Workspace
 ws	Guardar la sesión de esta carpeta (archivos abiertos, layout)	
 wr	Restaurar la sesión de esta carpeta Vuelves a un proyecto y recuperas todo como lo dejaste	

Comentarios & Surround		gc = go comment · s = surround
gcc	Comentar / descomentar la línea actual (toggle)	
gc + mov	Comentar por movimiento gc3j = 3 líneas · gcip = párrafo · gcG = hasta el final	
gc (visual) /  /	Comentar la selección En JSX/HTML el estilo de comentario se ajusta solo	
ys {mov}{c}	Rodear con comillas/paréntesis/tag ej. ysiw" → rodea la palabra con comillas: "palabra"	
ds {c} cs {a}{b}	Borrar envoltorio (ds") · Cambiar envoltorio (cs" pasa de " a ')	

Autocompletado (en modo insert)		nvim-cmp
^Space	Mostrar sugerencias manualmente	
^j / ^k	Bajar / subir en la lista de sugerencias	
 / 	Confirmar la sugerencia seleccionada	
^b / ^f ^e	Scroll de la documentación · cerrar el menú	

TMUX

Prefix = `Ctrl+a`. Casi todo empieza así: presionas `Ctrl+a`, **sueltras**, y luego la tecla. Abajo lo escribo como `^a` `tecla`.

🧠 Modelo mental — tmux tiene 3 niveles:

Sesión = un proyecto completo (ej. "kambista"). Sobrevive aunque cierres la terminal.

Window (ventana) = una pestaña dentro de la sesión (ej. editor / servidor / git).

Pane (panel) = una división visual dentro de una ventana.

Tus sesiones se **auto-guardan cada 15 min** y se **restauran solas** al reiniciar (resurrect + continuum).

Sesiones

lo que sobrevive al cerrar la terminal

<code>tmux new -s api</code>	Crear sesión con nombre (desde la terminal) Hazlo siempre con nombre para reconocerla luego
<code>tmux ls</code>	Listar sesiones
<code>tmux attach -t api</code>	Entrar a una sesión existente
<code>^a d</code>	Detach : salir dejando todo corriendo por detrás La tecla estrella de tmux. Cierras la terminal y nada se pierde
<code>^a s</code>	Árbol de sesiones (navega ↑↓, Enter entra) Úsalo para saltar entre sesiones SIN el error de "nesting"
<code>^a \$</code>	Renombrar la sesión
<code>^a (/)</code>	Sesión anterior / siguiente
<code>^a L</code>	Última sesión donde estuviste (toggle)
<code>tmux kill-server</code>	Matar todo y empezar de cero (desde la terminal)

Windows (pestañas)

<code>^a c</code>	Crear window (<i>c = create</i>)
<code>^a r</code>	Renombrar la window (pre-rellena el nombre actual) (<i>, también sirve</i>)
<code>^a 1 ... 9</code>	Ir a la window N (<i>las tuyas empiezan en 1</i>)
<code>^a h / l</code>	Anterior / siguiente window (minúscula = window). Repetible <i>n / p</i> por defecto también funcionan
<code>^a w</code>	Árbol de windows de todas las sesiones
<code>^a &</code>	Cerrar la window (pide confirmación)
<code>^a f</code>	Buscar window por texto

Panes (paneles)		donde vives el 80% del tiempo
<code>^a</code> <code>v</code>	Split vertical (lado a lado)	
<code>^a</code> <code>-</code>	Split horizontal (arriba / abajo)	
<code>^a</code> <code>x</code>	Cerrar el pane (sin confirmación)	
<code>^h</code> <code>^j</code> <code>^k</code> <code>^l</code>	Moverte entre panes — SIN prefix Las mismas teclas saltan también a los splits de Neovim	
<code>^a</code> <code>H/L</code> <code>j/k</code>	Redimensionar el pane: H/L (izq/der, MAYÚSCULA) · j/k (abajo/arriba). Repetible Regla: MAYÚSCULA = resize pane · minúscula h/l = cambiar window	
<code>^a</code> <code>m</code>	Zoom : el pane a pantalla completa; repite para volver Úsalo mucho para enfocarte en un pane	
<code>^a</code> <code>q</code>	Mostrar números de pane → presiona el número para saltar	
<code>^a</code> <code>Space</code>	Rotar layouts predefinidos	
<code>^a</code> <code>{</code> <code>/</code> <code>}</code>	Intercambiar pane con el anterior / siguiente	
<code>^a</code> <code>!</code>	Convertir el pane en una window propia	

Copy mode (scroll y copiar)		estilo vim
<code>^a</code> <code>C</code>	Entrar a copy mode (<i>[también sirve]</i>) · o solo scrollea con el mouse	
<code>h</code> <code>j</code> <code>k</code> <code>l</code> <code>/</code> <code>gg</code> <code>G</code>	Moverte como en vim (buscar con <code>/</code> , inicio <code>gg</code> , fin <code>G</code>)	
<code>v</code> → mover → <code>y</code>	Seleccionar y copiar al portapapeles de macOS Con <code>y</code> te quedas en copy mode; luego pegas con <code>Cmd+V</code> donde sea	
 arrastrar	Seleccionar con mouse → al soltar, copia al portapapeles y sale	
<code>q</code>	Salir de copy mode	

Config, plugins y ayuda	
<code>^a</code> <code>R</code>	Recargar la config (MAYÚSCULA; <code>r</code> pasó a renombrar window)
<code>^a</code> <code>I</code>	Instalar plugins nuevos (tpm) (<i>[mayúscula]</i>)
<code>^a</code> <code>U</code>	Actualizar plugins
<code>^a</code> <code>?</code>	Ver TODOS los atajos activos
<code>^a</code> <code>:</code>	Línea de comandos de tmux
<code>^a</code> <code>^s</code> <code>/</code> <code>^r</code>	Guardar / restaurar sesiones manualmente (resurrect)

Notas útiles de tmux: el **mouse está activado** (click para elegir pane, arrastra bordes para redimensionar, scroll, click en pestañas). Las windows y panes **empiezan en 1** y se **renumeran** al cerrar. Para mandar un **`Ctrl+a`** literal a un programa, presiónalo **dos veces**.

TERMINAL (zsh + WezTerm)

Tu shell es **zsh** en **WezTerm**. Estas teclas ya están configuradas para sentirse como macOS.

💡 **La sugerencia gris (autosuggestions)** aparece sola según tu historial mientras escribes — NO es lo mismo que buscar con ↑.

→ (**flecha derecha**) o **End**: aceptar TODA la sugerencia gris.

Option + →: aceptar solo UNA palabra de la sugerencia.

↑ / ↓: otra cosa — busca en el historial lo que empieza con lo que ya escribiste.

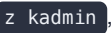
Edición de línea (estilo macOS)

 / 	Saltar por palabra (izquierda / derecha)
 / 	Inicio / fin de la línea
	Borrar hasta el inicio de la línea (como en macOS)
	Borrar la palabra anterior
	Borrar del cursor hasta el final
 / 	Inicio / fin (equivalente sin mouse)

Autosugerencias & historial

 / 	Aceptar la sugerencia gris completa
	Aceptar una palabra de la sugerencia
 / 	Buscar en historial por el prefijo que escribiste
	Búsqueda interactiva en todo el historial

Carpetas & archivos (zoxide + eza)

 (= z)	zoxide : salta a carpetas frecuentes →  , 
	Volver a la carpeta anterior
	Elegir carpeta en modo interactivo (fzf)
 (= eza)	Listar archivos con íconos

Control & limpieza

	Limpiar la pantalla (= clear)
	Cancelar el comando actual
	Cerrar la shell (EOF)
	Autocompletar comando / ruta

Tus aliases & scripts

`vim`abre **nvim**`eslint-init`

configura ESLint en el proyecto actual

`cheatsheet`

regenera este PDF

`denos`

deno run con permisos de env/net